

Implementação de um Sistema de Recomendação

1. Introdução

Este projeto tem como objetivo principal a **implementação de um sistema de recomendação** para clientes de lojas de varejo. Utilizando uma metodologia ativa, os alunos da disciplina de Programação Estruturada serão guiados através de uma série de atividades práticas que culminarão na construção de um sistema funcional. A abordagem visa não apenas o desenvolvimento de habilidades técnicas em programação, mas também a compreensão aprofundada dos conceitos teóricos por trás dos sistemas de recomendação.

2. Contexto do Projeto

Sistemas de recomendação são ferramentas essenciais no cenário atual do comércio eletrônico, auxiliando clientes a descobrir produtos que possam ser de seu interesse e, conseqüentemente, aumentando as vendas e a satisfação do consumidor. Eles funcionam analisando o comportamento de compra e as preferências dos usuários para sugerir itens relevantes.

2.1 Método da Vizinhança

Um dos métodos mais tradicionais e intuitivos para a construção de sistemas de recomendação é o Método da Vizinhança, que se baseia na ideia de que usuários com gostos ou comportamentos semelhantes tendem a gostar dos mesmos produtos. Este método pode ser dividido em duas abordagens principais: filtragem colaborativa baseada em usuário (*User-Based Collaborative Filtering*) e filtragem colaborativa baseada em item (*Item-Based Collaborative Filtering*). Neste projeto, focaremos na abordagem baseada em usuário.

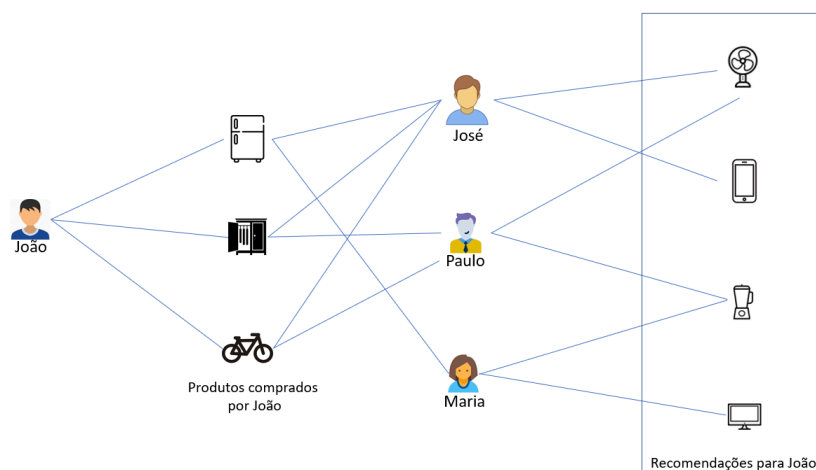


Figura 1. Ilustração do método da vizinhança. Recomendação de produtos para o cliente João baseada no histórico de compras de outros clientes.

Para aplicar o Método da Vizinhança, é fundamental representar as interações entre clientes e produtos. Isso é feito através de uma matriz de compras, onde as linhas representam os

clientes e as colunas representam os produtos. Uma entrada na matriz indica se um cliente comprou um determinado produto. A partir desta matriz, calculamos a similaridade entre clientes.

A distância de Jaccard será utilizada para medir a similaridade entre os clientes. Esta métrica é particularmente útil para conjuntos binários (como o histórico de compras, onde um produto foi comprado ou não) e é definida como o tamanho da interseção dividido pelo tamanho do conjunto de itens comprados por um dos clientes. Matematicamente, para dois clientes i e j , a similaridade de Jaccard é definida como:

$$s(i, j) = 1 - \frac{|P_i \cap P_j|}{|P_i|} . \quad (1)$$

Onde P_i e P_j são os conjuntos de produtos comprados pelos clientes i e j , respectivamente.

A Figura 2 ilustra um exemplo de relação das compras dos clientes João e Paulo. João comprou 3 produtos e Paulo comprou 4 produtos. Na interseção de produtos comprados temos dois produtos, logo, $|P_{joão} \cap P_{paulo}| = 2$.

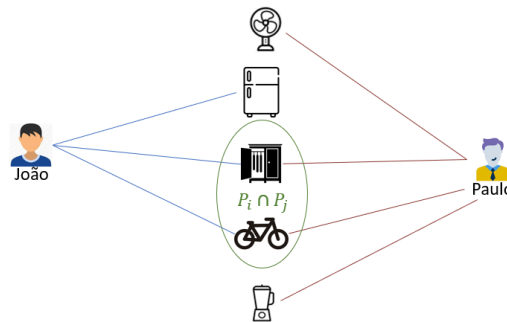


Figura 2. Relação de compras dos clientes João e Paulo e sua interseção.

Utilizando a equação (1), temos que $s(joão, paulo) = 1 - \frac{2}{3} = \frac{1}{3}$, enquanto $s(paulo, joão) = 1 - \frac{2}{4} = \frac{1}{2}$. Isso mostra que a matriz de similaridade é **assimétrica**, o que é desejável, pois evita que a presença de um grande comprador faça com que os demais sejam considerados muito semelhantes a ele, o que não refletiria a realidade.

2.2 Construção da Matriz de Similaridade

Considerando n como o número de clientes e m como o número de produtos, a matriz de compras $A_{n \times m}$ pode ser utilizada para construir a matriz de similaridade $S_{n \times n}$. A construção da matriz de similaridade passa pela construção da matriz de interseção $I_{n \times n}$ a partir da multiplicação da matriz de compras que é dada por:

$$I_{n \times n} = A_{n \times m} \times A_{m \times n}^T .$$

Onde $A_{m \times n}^T$ é a transposta da matriz de compras. Esta operação resulta em uma matriz onde cada elemento $I_{i,j}$ representa a interseção $P_i \cap P_j$ (ou o número de produtos em comum) entre o cliente i e o cliente j .

A Figura 3 ilustra a multiplicação do vetor de compras do cliente João pelos vetores de compras dos demais clientes da base, resultando no vetor de interseção das compras de João. Por exemplo, o produto escalar entre o vetor de compras de João e o de Maria é igual a 2, indicando que ambos os clientes adquiriram dois produtos em comum, especificamente os produtos B e F.

João		1	1			1	
------	--	---	---	--	--	---	--

	A	B	C	D	E	F	$ P_i \cap P_j $
João		1	1			1	3
Fulano	1		1				1
Ciclano				1	1		0
Maria		1	0	1	1	1	2

Figura 3. Computação do vetor de interseção de compras do cliente João ($I_{joão}$) com os outros clientes da base.

Por fim, para construir a matriz de similaridade $S_{n \times n}$ final, computamos cada célula da matriz como:

$$S_{i,j} \leftarrow 1 - \frac{I_{i,j}}{|P_i|}.$$

2.3 Algoritmo de Recomendação

O algoritmo de recomendação de produtos aos clientes se dá através do ranqueamento dos produtos com base na matriz de similaridade entre clientes. Para recomendar k produtos a um cliente c , o algoritmo segue os seguintes passos:

- 1 **Identificação de Vizinhos:** Crie uma lista L com todos os clientes que possuem similaridade com c (similaridade menor que 1 e que não seja o próprio cliente c);
- 2 **Inicialização do Vetor de Ranqueamento:** Para cada cliente, crie um vetor R de ranqueamento para todos os produtos da loja, atribuindo um valor inicial de 1 para cada produto;
- 3 **Cálculo do Ranqueamento:** Para cada cliente $s \in L$ e para cada produto p comprado por s e não comprado por c , atualize o ranqueamento específico do produto p fazendo: $R_p \leftarrow R_p \times s(c, s)$;
- 4 **Ordenação e Seleção:** Ordene o vetor R de forma não decrescente e retorne os k primeiros produtos da lista R . Estes serão os produtos melhor ranqueados e, portanto, recomendados ao cliente c .

3. Atividades do Projeto

Cada atividade é projetada para ser um módulo independente, mas interconectado, que contribui para a construção do sistema de recomendação completo. Cada atividade incluirá:

- **Descrição da Atividade:** Detalhamento do que precisa ser implementado.
- **Teoria Necessária:** Conceitos e algoritmos fundamentais para a implementação.
- **Entregáveis:** O que os alunos devem apresentar ao final da atividade.

Avaliação

A avaliação será contínua, baseada na entrega e funcionalidade de cada módulo, na clareza do código, na documentação interna e na capacidade de demonstrar a compreensão dos conceitos teóricos e práticos envolvidos. A performance e a otimização das implementações também serão critérios importantes, especialmente nas atividades que envolvem comparação de algoritmos e tratamento de matrizes esparsas.

Cronograma

Atividade	Data apresentação
Entregas A1, A2 e A3	21/07/2026
Entregas A4 e A5	28/07/2026
Entrega A6	04/08/2026

Atividade 1: Módulo ListaCompras

Descrição:

Implementação do módulo ListaCompras, responsável por organizar e armazenar o histórico de compras dos clientes. Será fornecido um histórico de compras como entrada, e o módulo deverá processá-lo para criar e preencher as seguintes estruturas de dados:

- Um **vetor** contendo os códigos de todos os clientes.
- Um **mapa** que associa o código original do cliente (da base da loja) a um índice interno único.
- Um **vetor** contendo os nomes de todos os produtos.
- Um **mapa** que associa o código original do produto (da base da loja) a um índice interno único do vetor de produtos.
- Um **vetor** com dimensão igual ao número de clientes, onde cada célula contém uma lista de identificadores internos dos produtos comprados pelo respectivo cliente.

A base de dados utilizada é um arquivo texto no formato CSV, que possui uma estrutura tabular, como ilustrado no exemplo a seguir:

DATA_COMPRA	COD_CLIENTE	COD_PRODUTO	NOME_PRODUTO
20200515	05090301	025728	SMARTPHONE SAMSUNG GAL J4 CORE 16GB PRETO
20200515	05190001	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO
20200515	99DL9N01	026243	SUPORTE UNIV. BEDIN BU-25
20200515	05992701	026574	SMARTPHONE SAMSUNG GAL A30S 64GB BRANCO
20200515	37578201	025367	TABLET NB729 MINI MS40G BRANCO
20200515	23210301	025728	SMARTPHONE SAMSUNG GAL J4 CORE 16GB PRETO
20200515	93168801	025354	TELEVISOR SMART 32" UN32J4290 SAMSUNG
20200515	77254401	022620	CELULAR RIU-I R200 DB BRANCO-ROSA
20200515	62982901	026584	SMARTPHONE MOTOROLA MOTO G8 PLUS 64GB CEREJA
20200515	61397001	021854	CELULAR BLU JENNY 2.8 T276 DB PRETO-AZUL
20200515	54509401	021647	DVD D-15 KARAOKE MONDIAL
20200515	95783001	025246	PEN DRIVE PD590 64GB TWIST PRETO
20200515	64669701	025996	TELEVISOR 32" 32S6500S HD LED SEMP TCL
20200515	99EOOG01	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO
20200515	99DQI601	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO

Cada linha representa o registro de uma compra individual e contém as seguintes colunas descritivas:

- **DATA_COMPRA:** data em que a compra foi realizada, no formato *aaaammdd*;
- **COD_CLIENTE:** código único que identifica o cliente no sistema de informação da loja;
- **COD_PRODUTO:** código único que identifica o produto no sistema de informação da loja;
- **NOME_PRODUTO:** nome descritivo do produto adquirido.

O preenchimento das estruturas ocorre em duas fases, conforme ilustrado na Figura 4.

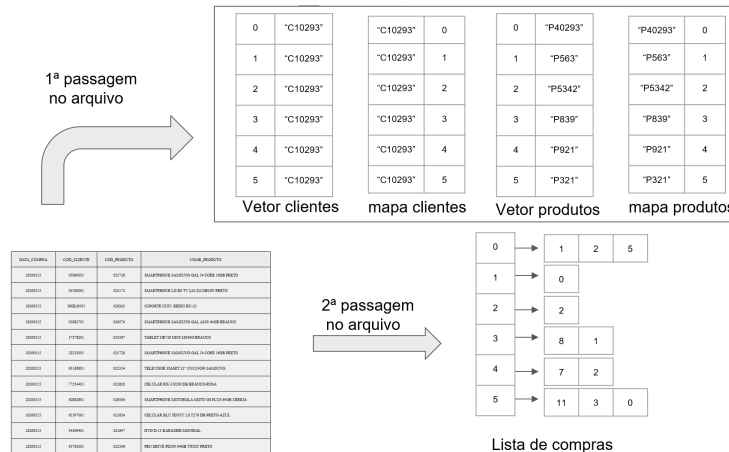


Figura 4. Esquema de leitura da base de compras e preenchimento das estruturas do módulo ListaCompras.

Na primeira fase, os códigos originais de cada cliente e produto são associados aos índices internos de controle por meio de vetores que armazenam esses códigos e de mapas reversos, os quais indicam, dado um código original, qual é o identificador interno correspondente.

Na segunda fase, é construída a lista de compras, composta por um vetor cuja dimensão corresponde ao número de clientes da base. Cada posição desse vetor contém uma lista dos índices internos dos produtos adquiridos pelo respectivo cliente.

Teoria aplicada:

Leitura de Arquivos CSV

Arquivos CSV (*Comma-Separated Values*) são arquivos de texto por linhas, e cada linha contém um conjunto de valores separados por vírgula.

Por exemplo, o arquivo chamado “dados.csv” pode conter o seguinte conteúdo:

```
Nome,Idade,Cidade
Ana,25,Recife
Bruno,30,Curitiba
Carla,22,Fortaleza
```

Cada linha representa um registro, e cada valor separado por vírgula é um campo do registro.

A função *fscanf()* é semelhante à função *scanf()*, mas em vez de ler dados do teclado, ela lê dados de um arquivo. Ela permite especificar o formato dos dados a serem lidos e armazená-los em variáveis apropriadas.

Código completo em C da leitura do arquivo “dados.csv”:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nome[50];
    int idade;
    char cidade[50];
} Pessoa;

int main() {
    FILE *arquivo;
    Pessoa pessoa;

    // Abre o arquivo CSV para leitura
    arquivo = fopen("dados.csv", "r");
    if (arquivo == NULL) {
        perror("Erro ao abrir o arquivo");
        return 1;
    }

    printf("Leitura dos dados do arquivo:\n");
    printf("-----\n");

    // Lê linha por linha usando fscanf
    while (fscanf(arquivo, "%49[^,],%d,%49[^\n]\n",
        pessoa.nome,
        &pessoa.idade,
        pessoa.cidade) == 3) {
        printf("Nome: %-10s | Idade: %2d | Cidade: %s\n",
            pessoa.nome, pessoa.idade, pessoa.cidade);
    }
    fclose(arquivo);
    return 0;
}
```

O programa abre o arquivo “dados.csv” para leitura e utiliza *fscanf()* para ler cada linha e extrair os valores separados por vírgula. Formatação de leitura dos campos:

- **%49[^,]** : lê até encontrar uma vírgula (usado para o nome e a cidade);
- **%d** : lê um número inteiro (a idade);
- **%49[^\n]** : lê até o final da linha.

Estrutura de dados STL

Para esta atividade, é fundamental compreender e aplicar as estruturas de dados da *Standard Template Library* (STL) do C++:

std::vector: estrutura dinâmica que armazena uma sequência de elementos contíguos na memória, mas cujo tamanho pode variar durante a execução do programa. Seu acesso a qualquer elemento é feito em tempo constante.

Código exemplo que utiliza o vector:

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Criação de um vetor dinâmico de inteiros
    vector<int> numeros;

    // Inserção de elementos
    numeros.push_back(10);
    numeros.push_back(20);
    numeros.push_back(30);

    // Acesso direto por índice
    cout << "Segundo elemento: " << numeros[1] << endl; // Saída: 20

    // Inserindo mais elementos dinamicamente
    numeros.push_back(40);
    numeros.push_back(50);

    cout << "Tamanho atual do vetor: " << numeros.size() << endl;

    return 0;
}

```

Principais funções do vector no código:

- **vector<int> numeros** : cria um vetor dinâmico vazio de inteiros.
- **push_back(valor)** : insere o valor no final do vetor.
- **numeros[i]** : acessa o elemento na posição *i*.
- **size()** : retorna a quantidade atual de elementos armazenados.

std::list: Uma lista encadeada é uma estrutura de dados linear, formada por uma sequência de elementos (nós), onde cada nó armazena um valor e um ponteiro para o próximo nó da sequência. Diferentemente de vetores, os elementos da lista não estão armazenados em posições contíguas na memória; eles são ligados entre si por referências (ponteiros). Sendo assim, a std::list não suporta acesso direto via índice (como lista[2]).

Código exemplo que utiliza o list:

```

#include <iostream>
#include <list>
using namespace std;

int main() {
    // Criação de uma lista de inteiros
    list<int> numeros;

    // Inserindo elementos no final da lista
    numeros.push_back(10);
    numeros.push_back(20);
    numeros.push_back(30);

    // Exibindo todos os elementos (iteração)
    cout << "Conteúdo da lista: ";
    for (int valor : numeros) {
        cout << valor << " ";
    }
    cout << endl;

    // Limpando a lista
    numeros.clear();
    cout << "Tamanho da lista após clear(): " << numeros.size() << endl;

    return 0;
}

```

Principais funções do vector no código:

- **list<int> numeros** : cria uma lista de inteiros vazia.
- **push_back(valor)** : insere um novo elemento no final da lista.
- **for (int valor : numeros)** : percorre a lista elemento por elemento (não há índices).
- **clear()** : remove todos os elementos da lista.

std::map: Um mapa, também conhecido como dicionário ou tabela de associação, é uma estrutura de dados que armazena pares de chave e valor (**key** → **value**). Cada chave é única dentro do mapa, e é usada para acessar o valor associado de forma eficiente. Não permite inserção de chaves duplicadas.

Código exemplo que utiliza o map:

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    // Criação de um mapa que associa nomes (chave) a idades (valor)
    map<string, int> idade;

    // Inserindo elementos no mapa
    idade["Ana"] = 25;
    idade["Bruno"] = 30;
    idade["Carla"] = 22;

    // Exibindo todos os pares (ordem crescente das chaves)
    for (auto& p : idade) {
        cout << p.first << " → " << p.second << endl;
    }

    // Pesquisando uma chave
    string nome = "Bruno";
    if (idade.find(nome) != idade.end()) {
        cout << nome << " está no mapa, idade = " << idade[nome] << endl;
    } else {
        cout << nome << " não encontrado." << endl;
    }

    // Verificando antes de inserir (para evitar sobrescrita)
    string novo_nome = "Ana";
    int nova_idade = 28;

    if (idade.find(novo_nome) == idade.end()) {
        idade[novo_nome] = nova_idade;
        cout << "Inserido: " << novo_nome << " → " << nova_idade << endl;
    } else {
        cout << "Aviso: " << novo_nome << " já está no mapa, não inserido novamente.\n";
        return 0;
    }
}
```

Principais funções do vector no código:

- **map<string, int> idade** : cria um mapa cujas chaves são strings (nomes) e os valores são int (idades).
- **idade["Ana"] = 25** : cria o par "Ana" → 25 se a chave ainda não existir. Sobrescreve o valor se a chave já existir.
- **find()** : retorna um iterador para o elemento, ou *end()* se a chave não for encontrada.

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo ListaCompras e um programa testador. O testador deve ser capaz de exibir os nomes dos produtos comprados por pelo menos 3 clientes, utilizando seus códigos originais como entrada.

Atividade 2: Módulo Similaridade (versão inicial)

Descrição:

Implementação do módulo Similaridade, que calcula e fornece o valor da função de similaridade entre dois clientes, conforme a definição da distância de Jaccard apresentada na Seção 2.2. A identificação dos clientes para este módulo deve ser interna (índices), não os códigos originais da base de compras. Nesta primeira versão, o módulo Similaridade receberá os dados do módulo ListaCompras, transformará esses dados em uma matriz de compras densa e, em seguida, computará a matriz de similaridade multiplicando a matriz de compras por sua transposta.

Teoria aplicada:

Algoritmo de Multiplicação de Matrizes Padrão:

A multiplicação de matrizes é uma operação que combina duas matrizes A e B para produzir uma terceira matriz C . Seja A de dimensão $m \times n$ e B de dimensão $n \times p$, então o produto $C = A \times B$ é uma matriz de dimensão $m \times p$.

Na multiplicação de matriz cada célula $C[i][j]$ é computada da seguinte forma:

$$C[i][j] = \sum_{k=1}^n A[i][k] \times B[k][j]$$

Pseudocódigo da multiplicação de matrizes.

```
Algoritmo MultiplicaMatrizes(A, B)
  Entrada:
    A[m][n] - matriz de dimensão m x n
    B[n][p] - matriz de dimensão n x p
  Saída:
    C[m][p] - matriz resultado do produto A x B

  Para i ← 1 até m faça
    Para j ← 1 até p faça
      C[i][j] ← 0
      Para k ← 1 até n faça
        C[i][j] ← C[i][j] + A[i][k] * B[k][j]
      FimPara
    FimPara
  FimPara

  Retorne C
FimAlgoritmo
```

Matriz Transposta:

A matriz transposta de uma matriz A , denotada por A^T , é obtida trocando suas linhas por colunas. Formalmente:

$$A^T[i][j] = A[j][i].$$

Ou seja, a primeira linha de A vira a primeira coluna de A^T ; a segunda linha vira a segunda coluna, e assim por diante.

Pseudocódigo da transposta da matriz:

```

Algoritmo Transposta(A)
  Entrada:
    A[m][n] - matriz de dimensão m x n
  Saída:
    T[n][m] - matriz transposta de A

  Para i ← 1 até m faça
    Para j ← 1 até n faça
      T[j][i] ← A[i][j]
    FimPara
  FimPara

  Retorne T
FimAlgoritmo

```

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo Similaridade e um programa testador. O testador deve ser capaz de, para dois clientes quaisquer (identificados por seus índices internos), encontrar e exibir o cliente mais similar que não seja ele mesmo.

Atividade 3: Módulo Recomendação

Descrição:

O módulo Recomendação será responsável por gerar as recomendações de produtos para cada cliente da loja, utilizando o algoritmo de ranqueamento descrito na Seção 2.3 deste projeto. O módulo deverá retornar os k produtos mais bem ranqueados para um determinado cliente.

O vetor de ranqueamento de cada cliente deve ser composto por uma estrutura contendo o identificador interno do produto e o respectivo valor de ranqueamento. Esse vetor deve ser ordenado por meio de um algoritmo de ordenação, tomando como base o valor de ranqueamento de cada produto.

Teoria aplicada:

Algoritmos de Ordenação

Os algoritmos de ordenação são métodos utilizados para reorganizar elementos de uma coleção (como vetores, listas ou registros) segundo uma determinada ordem, normalmente crescente ou decrescente, com base em algum critério de comparação.

Ordenar dados é uma das tarefas mais importantes em computação, pois:

- Melhora a eficiência de busca (por exemplo, em uma busca binária);
- Facilita a análise de dados e a visualização de padrões.

Código exemplo em C para ordenar um vetor registros de uma estrutura personalizada:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

struct Produto {
    std::string nome;
    double preco;
};

// Função de comparação: retorna true se 'a' deve vir antes de 'b'
bool compararPorPreco(const Produto &a, const Produto &b) {
    return a.preco < b.preco; // ordem crescente de preço
}

int main() {
    std::vector<Produto> produtos = {
        {"Teclado", 150.0},
        {"Mouse", 80.0},
        {"Monitor", 900.0}
    };

    // Ordena o vetor usando a função de comparação
    std::sort(produtos.begin(), produtos.end(), compararPorPreco);

    std::cout << "Produtos ordenados por preço:\n";
    for (const auto &p : produtos)
        std::cout << p.nome << " - R$ " << p.preco << std::endl;

    return 0;
}
```

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo Recomendação e um programa testador. O testador deve solicitar a recomendação para 3 clientes, utilizando seus códigos originais, e exibir os nomes dos k produtos melhor recomendados para cada cliente, demonstrando a funcionalidade completa do sistema de recomendação.

Atividade 4: Multiplicação Eficiente

Descrição:

Nesta atividade, os alunos deverão reimplementar o algoritmo de multiplicação de matrizes dentro do módulo Similaridade utilizando o fato de que a matriz de compras é multiplicada por sua própria transposta. Eliminaremos a necessidade de fazer a transposta e utilizaremos o fato da matriz de similaridade produto da operação ser similar, ou seja, $S[i][j] = S[j][i]$. Ao final, será necessário demonstrar experimentalmente que o algoritmo adaptado é mais rápido computacionalmente.

Teoria aplicada:

Algoritmo de Multiplicação melhorado

Uma otimização clássica e muito importante quando queremos calcular

$$C = A \times A^T$$

sem realmente construir a transposta A^T na memória, o que seria caro em termos de tempo e espaço, especialmente para matrizes grandes ou esparsas.

Para isto, basta calcularmos

$$C[i][j] = \sum_{k=1}^n A[i][k] \times A[j][k].$$

Pseudocódigo adaptado

```
Entrada: matriz A de dimensão n × m
Saída: matriz C de dimensão n × n, onde C = A × A^T

função multiplica_por_transposta(A):
    n ← número de linhas de A
    m ← número de colunas de A
    C ← matriz n × n inicializada com zeros

    para i de 0 até n-1:
        para j de i até n-1:           // usa simetria: C[i][j] = C[j][i]
            soma ← 0
            para k de 0 até m-1:
                soma ← soma + A[i][k] * A[j][k]
            fin para
            C[i][j] ← soma
            C[j][i] ← soma           // aproveita a simetria
        fin para
    fin para

    retornar C
```

Como C é simétrica, calculamos apenas a metade superior de C (onde $i \leq j$), reduzindo o número de multiplicações quase pela metade.

Aspecto	Algoritmo padrão ($A \times A^T$)	Algoritmo adaptado
Memória	Cria cópia transposta	Usa apenas A
Operações	$(n^2 \times m)$	$(\frac{n^2 \times m}{2})$ (com simetria)
Aplicação	Densa e esparsa	Ideal para ambos

Medindo o tempo de execução

O objetivo é registrar o tempo no início e no final de um bloco de código, e depois calcular a diferença.

Isso permite saber quanto tempo o algoritmo demorou para ser executado — geralmente em milissegundos (ms) ou segundos (s).

Para isto utilizamos a função `clock()` da biblioteca `<ctime>`. Ela mede o tempo de CPU gasto pelo processo, não o tempo real de parede (“wall clock”).

Pseudocódigo

```
#include <iostream>
#include <ctime> // biblioteca tradicional de tempo
#include <thread>

int main() {
    clock_t inicio = clock();

    // --- Bloco de código a medir ---
    std::this_thread::sleep_for(std::chrono::milliseconds(500)); // simula
    processamento
    // -----

    clock_t fim = clock();

    double tempo = double(fim - inicio) / CLOCKS_PER_SEC; // converte para
    segundos

    std::cout << "Tempo de execucao: " << tempo << " segundos\n";

    return 0;
}
```

- `clock()` → retorna o número de “ticks” de CPU desde o início do programa.
- `CLOCKS_PER_SEC` → constante que converte ticks para segundos.

Entregáveis:

Os alunos deverão entregar o código-fonte atualizado do módulo Similaridade com a implementação do algoritmo de multiplicação adaptado. Um parâmetro deve ser adicionado ao módulo para permitir a seleção entre o algoritmo padrão e o adaptado. Além disso, um teste comparativo deve ser realizado, medindo e exibindo o tempo de execução da construção da matriz de similaridade utilizando ambas as versões do algoritmo para diferentes tamanhos de matrizes.

Atividade 5: Matrizes Esparsas (CSR)

Descrição:

Como discutido anteriormente, as dimensões das matrizes de compras e de similaridade são, respectivamente, $n \times m$ e $n \times n$. Ao analisar a base de dados real de um ano de uma loja de varejo, com $n = 592.000$ clientes e $m = 3.500$ produtos, observa-se que a matriz de compras densa ocuparia cerca de 15 GB de memória, enquanto a matriz de similaridade densa exigiria mais de 2,5 TB de RAM.

Entretanto, a grande maioria desses 592 mil clientes adquiriu apenas alguns poucos produtos distintos, como ilustrado na Figura 5. Apenas 42% dos clientes compraram dois ou mais produtos, e somente 0,68% realizaram compras de dez ou mais produtos. Consequentemente, a maioria dos elementos da matriz de compras é igual a zero — por exemplo, apenas 0,054% dos elementos da matriz possuem valor 1.

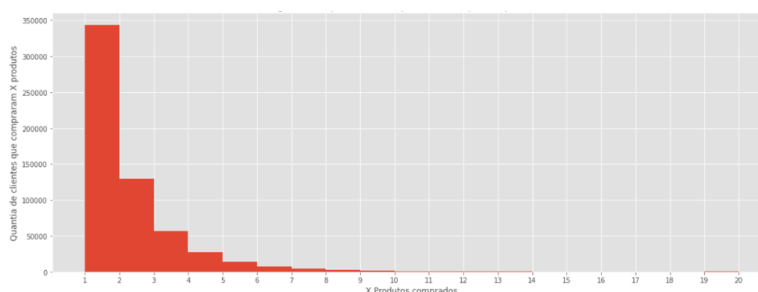


Figura 5. Histograma da quantidade de produtos comprados pelos clientes.

Essa matriz de compras é, portanto, caracterizada como esparsa. De modo geral, dizemos que uma matriz é esparsa quando a proporção de elementos nulos é muito maior que a de elementos não nulos.

Para representar esse tipo de matriz no computador, a principal ideia consiste em armazenar apenas os elementos diferentes de zero e suas posições. Essa estratégia permite:

- 5 Reduzir drasticamente o uso de memória;
- 6 Acelerar as operações matemáticas, já que os elementos nulos não precisam ser processados.

Esta atividade concentra-se na implementação de um algoritmo de multiplicação de matrizes esparsas e em sua adaptação para a construção da matriz de similaridade. Considerando que tanto a matriz de compras quanto a matriz de similaridade podem ser extremamente grandes e esparsas (com grande quantidade de valores zero), a utilização de estruturas de dados otimizadas para matrizes esparsas é essencial para garantir eficiência em memória e processamento.

Teoria aplicada:

Estrutura CSR (Compressed Sparse Row)

A estrutura CSR é um formato de armazenamento que mantém apenas os elementos não nulos da matriz, juntamente com as posições de suas colunas e os índices que marcam o início de cada linha.

Ela usa três vetores principais:

Vetor	Descrição
values	Armazena os valores não nulos da matriz, linha por linha.
col_index	Guarda o índice da coluna correspondente a cada valor em values.
row_ptr	Indica o início de cada linha no vetor values. Tem tamanho (n_linhas + 1).

O vetor *row_ptr* é a “cola” que permite saber quais elementos pertencem a cada linha, sem precisar armazenar zeros.

Considere a matriz exemplo *A*:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 5 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}$$

Os vetores da matriz *A* na estrutura CSR são:

Vetor	Conteúdo	Descrição
values	[5, 8, 3]	Valores não nulos
col_index	[0, 2, 1]	Índices das colunas correspondentes
row_ptr	[0, 0, 2, 2, 3]	Índices de início das linhas no vetor values

Interpretação de *row_ptr*:

→ Linha 0 → começa em posição 0, termina em 0 → linha vazia
→ Linha 1 → começa em 0, termina em 2 →

contém 2 elementos

→ Linha 2 → começa em 2, termina em 2 → linha vazia

→ Linha 3 → começa em 2, termina em 3 → contém 1 elemento

Multiplicação $A \times A^T$ na estrutura CSR

Computar explicitamente a transposta A^T no formato CSR requer um algoritmo de implementação complexa e custosa. Assim, para realizar o produto da matriz *A* por sua transposta, adota-se a abordagem já discutida anteriormente: calcular o produto interno entre as linhas de *A*. Essa estratégia é especialmente conveniente, pois o formato CSR é naturalmente orientado à iteração sobre linhas, o que nos permite simular a iteração sobre as colunas de A^T sem a necessidade de construí-la.

Para ilustrar essa versão do produto $A \times A^T$, apresentaremos o passo a passo utilizando exemplos gráficos. Para isto, considere que a matriz resultante *C* também será armazenada no

formato CSR. Além disso, definimos a variável *cont* como o contador de elementos não nulos que serão inseridos em *C*, iniciando com *cont* = 0.

A figura a seguir apresenta uma matriz exemplo *A* e sua representação no formato CSR, bem como o cálculo do produto interno entre a linha *i* = 0 de *A* e a linha *j* = 0 (da matriz A^T). Observando o vetor *row_ptr*, nota-se que os valores correspondentes às linhas *i* e *j* estão contidos nos índices de 0 até 1 (isto é, antes de 2).

Como as colunas das linhas *i* e *j* coincidem, seus valores são multiplicados e somados na variável *soma*, que representa o resultado do produto interno. Como o valor de *soma* é maior que zero, a matriz resultado *C* é atualizada:

- o vetor *values* recebe *soma* na posição indicada por *cont*;
- o vetor *col_index* recebe o valor *j*, que representa a coluna simulada de A^T .

Ao final do processamento dessa operação, a variável *cont* é incrementada para registrar o próximo elemento não nulo de *C*.

$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 0 & 4 & 5 \end{bmatrix}$ values = 1, 2, 3, 4, 5 col_index = 0, 2, 1, 1, 2 row_ptr = 0, 2, 3, 5 $\underbrace{\hspace{1.5cm}}_{i,j}$	$i = j = 0$ soma = values[0] × values[0] + values[1] × values[1] Adição do produto na estrutura CSR da matriz C soma > 0 $\begin{cases} C_values[cont] = soma \\ C_col_index[cont] = j \\ cont = cont + 1 \end{cases}$
--	--

Na próxima figura, ilustramos o produto interno entre a linha *i* = 0 da matriz *A* e a linha *j* = 1 (da matriz A^T). Observando o vetor *row_ptr*, nota-se que os valores correspondentes à linha *i* estão armazenados nas posições 0 e 1 do vetor *values*, enquanto os valores da linha *j* encontram-se na posição 2.

Ao comparar os índices de coluna (*col_index*), verifica-se que: *col_index*[0] < *col_index*[2] e *col_index*[1] > *col_index*[2]. Isso significa que não há colunas em comum entre as linhas *i* e *j*. Consequentemente, o produto interno resulta em zero (*soma* = 0), não havendo, portanto, nenhum elemento a ser adicionado à matriz resultado *C*.

$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 0 & 4 & 5 \end{bmatrix}$ values = 1, 2, 3, 4, 5 col_index = 0, 2, 1, 1, 2 row_ptr = 0, 2, 3, 5 $\underbrace{\hspace{1.5cm}}_{i \quad j}$	$i = 0 \quad j = 1$ col_index[0] ≠ col_index[2] col_index[1] ≠ col_index[2] Como soma == 0 Sem atualização na estrutura CSR da matriz C
--	---

A figura a seguir ilustra o produto interno entre a linha $i = 0$ da matriz A e a linha $j = 2$ (da matriz A^T). Observando o vetor row_ptr , nota-se que os valores correspondentes à linha i permanecem os mesmos, enquanto os valores da linha j estão armazenados nas posições 3 e 4 do vetor $values$.

Ao comparar os índices de coluna (col_index), observa-se o seguinte:

- Inicialmente, $col_index[0] < col_index[3]$; portanto, avançamos o índice coluna da linha i ;
- Em seguida, $col_index[1] > col_index[3]$; assim, avançamos o índice da linha j ;
- Agora, $col_index[1] == col_index[4]$; portanto, encontramos uma coluna em comum, e o valor parcial do produto interno é $soma = values[1] \times values[4]$.

Como soma é diferente de zero, o valor é adicionado à matriz resultado C conforme o procedimento descrito anteriormente. Esta é a última iteração da linha i ; portanto, o vetor row_ptr deve ser atualizado para indicar a quantidade total de elementos não nulos presentes na linha i da matriz C .

$A = \begin{vmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 0 & 4 & 5 \end{vmatrix}$ values = 1, 2, 3, 4, 5 col_index = 0, 2, 1, 1, 2 row_ptr = 0, 2, 3, 5 i j		i = 0 j = 2 $col_index[0] \neq col_index[3]$ $col_index[1] \neq col_index[3]$ $soma = values[1] \times values[4]$ Adição do produto na estrutura CSR da matriz C $soma > 0 \begin{cases} C_values[cont] = soma \\ C_col_index[cont] = j \\ cont = cont + 1 \end{cases}$ $row_ptr[i + 1] = cont$
---	--	--

Por fim, após iterar sobre todas as linhas da matriz A , obtemos a matriz produto C totalmente construída, resultante da multiplicação $A \times A^T$. A figura a seguir apresenta sua representação no formato CSR.

$C = \begin{vmatrix} 5 & 0 & 10 \\ 0 & 9 & 12 \\ 10 & 12 & 41 \end{vmatrix}$		values = 5, 10, 9, 12, 10, 12, 41 col_index = 0, 2, 1, 2, 0, 1, 2 row_ptr = 0, 2, 4, 7
--	--	---

Entregáveis:

Os alunos deverão implementar a solução com matriz esparsa CSR e substituir o componente de similaridade entre clientes no módulo Similaridade para utilizar esta nova representação. Um testador deve ser criado para executar a construção da matriz de similaridade com uma instância real de uma loja de varejo e demonstrar a economia de memória e/ou o ganho de desempenho em comparação com a abordagem densa.

Atividade 6: Integração Python-C para o Sistema de Recomendação

Descrição:

Esta atividade tem como objetivo integrar as implementações em C++ dos módulos de recomendação, desenvolvidos nas etapas anteriores, com a linguagem Python. Os alunos deverão reproduzir a funcionalidade do sistema implementado na Atividade 1 em Python, mas utilizando os módulos de recomendação (Atividades 2, 3 e 4) já implementados em C++. Para isso, um módulo em Python precisará fazer o *bind* (ligação) com o código em C++.

Teoria aplicada:

O Python é amplamente reconhecido como uma das linguagens mais importantes para prototipagem rápida e análise de dados, devido à sua simplicidade sintática, leitura intuitiva e enorme ecossistema de bibliotecas. Python oferece um ciclo de desenvolvimento extremamente produtivo:

- **Ecossistema poderoso**: bibliotecas como *NumPy*, *Pandas*, *Matplotlib*, *Scikit-learn*, *TensorFlow* e *PyTorch* fornecem ferramentas robustas para manipulação de dados, aprendizado de máquina, visualização e cálculo científico;
- **Interatividade**: ambientes como o Jupyter Notebook permitem testar ideias iterativamente, visualizar resultados e documentar o processo de forma integrada.

No entanto, **essa flexibilidade tem um custo**. Python é uma linguagem interpretada e de alto nível, o que significa que certas operações, especialmente loops e cálculos numéricos intensos, podem ser muito mais lentas do que em linguagens de baixo nível como C ou C++, que oferecem controle direto sobre memória e compilação otimizada. Para superar essa limitação, utiliza-se o conceito de binding (ou ligação) entre Python e C/C++. Essa técnica permite que funções escritas em C ou C++, que são extremamente rápidas, sejam acessadas e executadas dentro do Python como se fossem funções nativas.

Como Realizar o Bind de Código C/C++ para Python?

A integração entre Python e C/C++ é uma prática comum em aplicações que demandam alto desempenho computacional aliado à flexibilidade de linguagens de alto nível. Nesse contexto, o código crítico (mais custoso computacionalmente) é implementado em C/C++, enquanto o Python é utilizado como interface de controle, visualização e experimentação. Existem diferentes abordagens para realizar essa integração, dentre as quais se destacam:

- ***ctypes***: biblioteca padrão do Python que permite carregar bibliotecas compiladas (DLL/so) e chamar funções diretamente;
- ***SWIG***: ferramenta automática que gera interfaces entre C/C++ e várias linguagens, incluindo Python;
- ***Cython***: linguagem intermediária que combina Python e C para gerar código compilado;
- ***pybind11***: biblioteca moderna em C++ que permite expor funções e classes C++ para Python de forma simples e segura.

Neste projeto, será utilizada a biblioteca *pybind11*, por apresentar:

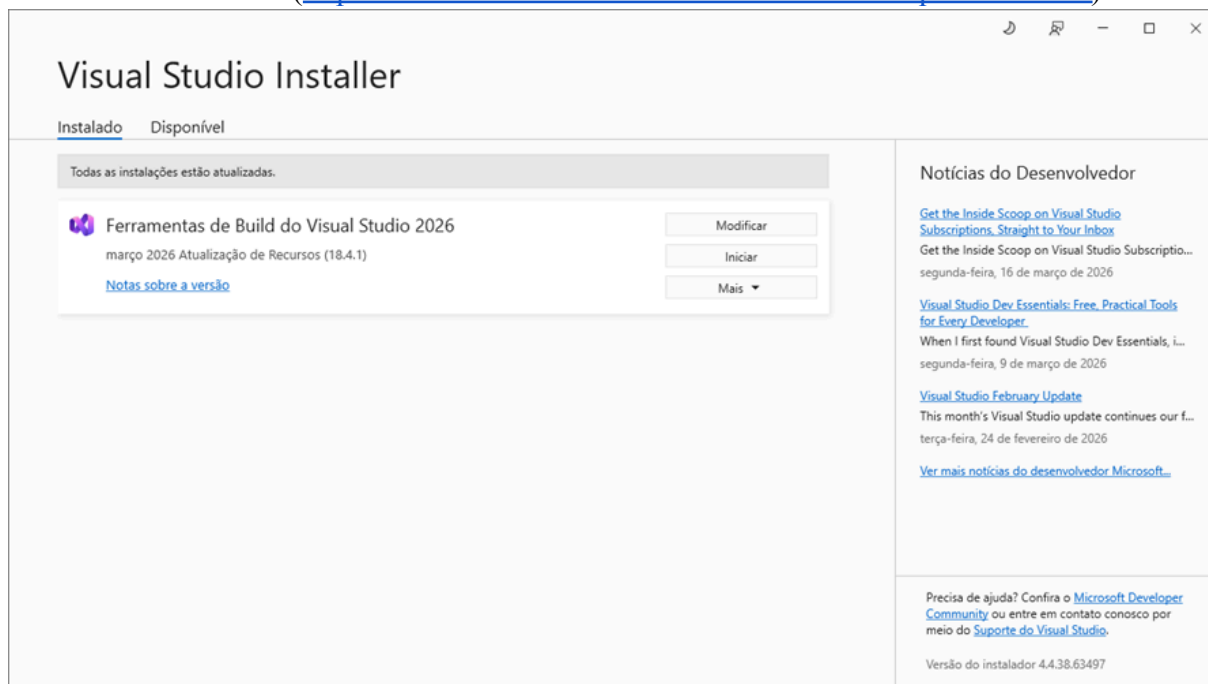
- Sintaxe simples e próxima do C++;

- Conversão automática de tipos entre Python e C++;
- Integração direta com estruturas da STL (como vector, tuple, etc.);
- Baixa sobrecarga e alto desempenho.

Instalação e Integração:

Windows

1. Instalar Python
2. Instalar VS Code
3. Instalar o compilador C++ da Microsoft
 - Para compilar o módulo C++, não basta ter só o Python. É necessário instalar
 - Microsoft C++ Build Tools
(<https://visualstudio.microsoft.com/visual-cpp-build-tools/>) ou
 - Visual Studio com C++
(<https://visualstudio.microsoft.com/downloads/?q=build+tools>)



Durante a instalação, é obrigatório marcar a opção Desenvolvimento para desktop com C++ (*Desktop development with C++*) que vai estar disponível indo em modificar. Essa opção instala os componentes necessários, incluindo: compilador MSVC, bibliotecas C++ e Windows SDK.

4. Instalar bibliotecas Python: `pip install pybind11 setuptools wheel`
5. Organizar o projeto

```
integracao/
|
├── dados_clientes.txt
├── leitura_e_execucao.py
├── solver.cpp
└── setup.py
```

Onde:

- O arquivo *dados_clientes.txt* contém a instância a ser resolvida;
 - *leitura_e_execucao.py* contém o código que realiza a leitura da instância e que chama as funções implementadas em C++
 - *solver.cpp* é o arquivo onde as funções de C++ estão implementadas
 - *setup.py* contém comandos para integração dos códigos;
6. Para compilar o código, deve-se, de preferência, usar *x64 Native Tools Command Prompt for Visual Studio* ou *Developer PowerShell for Visual Studio (x64)*, apesar de o PowerShell comum poder ser usado.
7. Para compilar o código, deve-se:
- Entrar na pasta do projeto: `cd C:\caminho\para\integracao`
 - Executar: `python setup.py build_ext --inplace`
 - Se tudo estiver ok, será criar um arquivo: *meu_solver.cp313-win_amd64.pyd* (meu solver foi o nome definido dentro de [setup.py](#))
 - Para executar o programa, execute: `python leitura_e_execucao.py`

Estrutura Geral do Processo

O processo de integração com *pybind11* envolve três etapas principais:

1. **Implementação das funções em C/C++:** Nas etapas anteriores, os módulos *ListaCompras*, *Similaridade* e *Recomendação* foram implementados em C/C++. Essas funções serão reutilizadas diretamente.
2. **Criação do módulo Python em C++:** É necessário criar um arquivo adicional (por exemplo, *bindings.cpp*) contendo o código de integração com o *pybind11*.

```

#include <pybind11/pybind11.h>
#include <pybind11/stl.h>

namespace py = pybind11;

// Função C++ já existente
double soma(double a, double b) {
    return a + b;
}

// Criação do módulo Python
PYBIND11_MODULE(meu_modulo, m) {
    m.doc() = "Exemplo simples de integração C++/Python";

    m.def("soma", &soma, "Soma dois números");
}

```

3. **Criação do módulo Python em C++:** O código deve ser compilado como uma biblioteca compatível com Python. Após a compilação, será gerado um arquivo (por exemplo, *meu_modulo.pyd* no Windows). Esse arquivo poderá ser importado diretamente no Python:

```
import meu_modulo

resultado = meu_modulo.soma(2, 3)
print(resultado) # Saída: 5
```

Integração com pybind11

O *pybind11* é uma biblioteca que permite criar uma ponte direta entre C++ e Python. Ele possibilita que funções e classes escritas em C++ sejam utilizadas como se fossem funções nativas do Python. A integração ocorre através da definição de um módulo, no qual funções C++ são registradas e disponibilizadas ao Python.

Definição do módulo

A criação do módulo é feita com a macro:

- Em C++

```
PYBIND11_MODULE(nome_modulo, m)
```

- *nome_modulo*: nome que será usado no Python (import nome_modulo);
- *m*: objeto que representa o módulo.

Exemplo:

- Em C++

```
PYBIND11_MODULE(meu_solver, m) {
    m.doc() = "Módulo de
recomendação";
}
```

Registro de funções

Para expor funções C++ ao Python, utiliza-se:

- Em C++

```
m.def("nome_python", &funcao_cpp);
```

Exemplo:

- Em C++

```
double soma_demandas( ... ) {  
    // implementação  
}  
  
m.def("soma_demandas",  
    &soma_demandas);
```

- No Python:

```
import meu_solver  
meu_solver.soma_demandas( ... )
```

Argumentos nomeados

É possível definir nomes para os argumentos utilizando:

- C++:

```
py::arg("nome")
```

Exemplo:

- Em C++

```
m.def("filtrar",  
    &clientes_demanda_maior_que,  
    py::arg("dados"),  
    py::arg("limite"));
```

- No Python

```
meu_solver.filtrar(dados=dados,  
    limite=10.0)
```

Conversão automática de tipos entre Python e C++

O *pybind11* fornece conversão automática entre diversos tipos nativos do Python e estruturas da STL (Standard Template Library) do C++.

- Python

```
dados = [(1, 10.0, 20.0, 5.0)]
```

- C++

```
std::vector<std::tuple<int, double,  
double, double>>
```

Essa funcionalidade é habilitada principalmente pelo include em C++:

```
#include <pybind11/stl.h>
```

A tabela a seguir apresenta as principais correspondências de tipos:

- Tipos básicos

Python	C++	Observação
<i>int</i>	<i>int</i>	<i>Conversão direta</i>
<i>float</i>	<i>double</i>	<i>Python não diferencia float/double</i>
<i>bool</i>	<i>bool</i>	<i>Conversão direta</i>
<i>str</i>	<i>std::string</i>	<i>Codificação UTF-8</i>
<i>None</i>	<i>nullptr / std::optional</i>	<i>Depende do uso</i>

- Estrutura de dados

Python	C++	Observação
<i>list</i>	<i>std::vector<T></i>	<i>Conversão mais comum</i>
<i>list</i>	<i>std::list<T></i>	<i>Funciona, mas menos eficiente</i>
<i>tuple</i>	<i>std::tuple<T1, T2, ...></i>	<i>Tamanho fixo</i>

<i>dict</i>	<i>std::map<K, V></i>	<i>Chaves devem ser compatíveis</i>
<i>set</i>	<i>std::set<T></i>	<i>Elementos únicos</i>

- Exemplos práticos

1. *list* -> *std::vector*

- C++

```
std::vector<int> dobrar(const
std::vector<int>& v) {
    std::vector<int> resultado;
    for (int x : v) {
        resultado.push_back(2 * x);
    }
    return resultado;
}
```

- Python

```
import meu_modulo

print(meu_modulo.dobrar([1, 2, 3]))
# Saída: [2, 4, 6]
```

2. *list* -> *std::list*

- C++

```
#include <list>

int soma_lista(const std::list<int>&
valores) {
    int soma = 0;
    for (int v : valores) {
        soma += v;
    }
    return soma;
}
```

- Python

```
meu_modulo.soma_lista([1, 2, 3, 4])
# Saída: 10
```

3. *tuple* -> *std::tuple*

- C++

```
std::tuple<int, double> exemplo() {  
    return std::make_tuple(1, 3.14);  
}
```

- Python

```
a, b = meu_modulo.exemplo()  
print(a, b)  
# Saída: 1 3.14
```

4. *dict* -> *std::map*

- C++

```
#include <map>  
  
std::map<std::string, int> contar() {  
    return {{ "Ana", 1}, {"Bruno",  
2}};  
}
```

- Python

```
d = meu_modulo.contar()  
print(d["Ana"]) # 1
```

5. *list de tuple* -> *vector de tuple*

- C++

```
dados = [  
    (1, 10.0, 20.0, 5.0),  
    (2, 15.0, 25.0, 8.0)  
]
```

- Python

```
std::vector<std::tuple<int, double,  
double, double>>
```

- Exemplo aplicado ao projeto.

Considere uma função do sistema de recomendação:

```
std::vector<int>
recomendar_produtos(int cliente_id,
int k);
```

Binding

- Em C++

```
m.def("recomendar_produtos",
      &recomendar_produtos,
      "Retorna os k produtos
recomendados",
      py::arg("cliente_id"),
      py::arg("k"));
```

- Em Python

```
import meu_solver

recomendacoes =
meu_solver.recomendar_produtos(client
e_id=3, k=5)
print(recomendacoes)
```

Entregáveis:

Os alunos deverão implementar o mesmo testador das Atividades 1-4, mas desta vez em Python.

- Este testador Python deve utilizar a API de recomendação que faz a amarração com os módulos em C++.
- Isso demonstrará a capacidade de integrar componentes de alto desempenho escritos em C++ com a flexibilidade e o ecossistema do Python, um cenário comum em aplicações de ciência de dados e inteligência artificial.
- Será necessário reproduzir a funcionalidade do sistema implementado na Atividade 1 (leitura de dados, Lista de Compras, etc.) em Python e utilizar os módulos de recomendação (Atividades 2, 3 e 4) implementados em C++ para realizar a recomendação.